

Prueba Técnica - Software Engineer

duppla

Contexto

duppla es una fintech colombiana que democratiza el acceso a vivienda mediante rent-to-own. Necesitamos evaluar tu capacidad para construir APIs robustas y escalables.

Tiempo estimado: 6-8 horas (usa IA libremente)

Entrega: Repositorio Git público + README

El Reto: API de Documentos Financieros

Construye una **API REST** que gestione documentos financieros (facturas, recibos, comprobantes).

Features Core

- 1. CRUD Básico**
 - Crear/leer/actualizar documentos
 - Campos: `id`, `tipo`, `monto`, `estado`, `fecha_creacion`, `metadata` (JSON)
- 2. Búsqueda con Filtros**
 - Por rango de fechas, tipo, estado, monto
 - Paginación obligatoria
- 3. Procesamiento Asíncrono**
 - Endpoint POST `/documentos/batch/procesar` que recibe un array de IDs
 - Simula procesamiento pesado (sleep 5-10s)
 - Retorna `job_id` inmediatamente
 - Endpoint GET `/jobs/{job_id}` para consultar estado
- 4. Sistema de Estados**
 - Workflow: `borrador` → `pendiente` → `aprobado/rechazado`
 - Validar transiciones (no saltar estados)

Se valorará la implementación proactiva de mecanismos de resiliencia y seguridad

Frontend Integration (Opcional - Plus de Candidato Senior)

En **duppla** operamos con React y Angular. Para evaluar tu capacidad de integración *fullstack*, puedes añadir una interfaz minimalista (una sola vista) usando cualquiera de los siguientes frameworks (React + Tailwind o Angular + ngBootstrap) que permita:

- **Visualizar el listado:** Una tabla simple que consuma `/documentos` con su paginación.
- **Trackear el procesamiento:** Un botón para disparar el `batch/procesar` y un indicador visual (spinner o barra de progreso) que haga *polling* al endpoint de `jobs` hasta que el estado sea "completado".
- *No buscamos un diseño perfecto, sino una implementación limpia de consumo de estados asíncronos.*

Stack

Usa lo que quieras. Si quieres alinearte con nosotros:

- Python/FastAPI
- PostgreSQL o SQLite
- Redis (Opcional)

Entregables

1. Código

- Git con commits atómicos (no un mega commit)
- `.env.example` con variables necesarias

2. README.md (CRÍTICO)

None

```
## Decisiones de Arquitectura
- ¿Por qué elegiste X?
- ¿Qué patrones usaste? (ej: Repository, Service Layer)
- ¿Qué mejoras de performance/seguridad implementaste?

## Setup
```bash
Comandos exactos para correr
```

```
docker-compose up -d
npm install && npm run dev
```

## Testing

- Ejemplo de curl/Postman para cada endpoint
- ¿Cómo probar el flujo completo?

## Trade-offs y Limitaciones

- ¿Qué faltó por tiempo?
- ¿Qué harías diferente en producción?

None

```
3. Docker Compose (Recomendado)
```yaml
# Ejemplo mínimo
services:
  db:
    image: postgres:15
  redis:
    image: redis:7-alpine
  api:
    build: .
    depends_on: [db, redis]
```

4. Tests (Mínimo 3-5)

- Happy path del CRUD
- Validación de estados inválidos
- Rate limiting funcionando

Evaluación

Criterio

%

Qué buscamos

Arquitectura	30%	Separación de capas, escalabilidad, principios SOLID
Features "Ocultos"	25%	-
Código Limpio	20%	Legibilidad, nomenclatura consistente
Funcionalidad	15%	¿Funciona según los requisitos?
Testing	10%	Cobertura básica, casos edge

Quick Tips

Está bien:

- Usar ChatGPT/Copilot (es 2026, seamos realistas)
- MVP bien hecho > arquitectura perfecta incompleta
- Preguntar si algo no está claro

Evita:

- Copy-paste sin entender
 - Sobre-ingeniería (no necesitamos microservicios)
 - README vacío o genérico
-

Bonus (100% Opcional)

- Autenticación básica (API Key header)
 - OpenAPI/Swagger docs
 - GitHub Actions con tests
 - Webhook al completar job (webhook.site)
 - Frontend Básico
-

Entrega

7 días desde hoy. Envía repo a juanma@duppla.co con asunto: "Prueba Técnica - [Tu Nombre]"

Siguiente paso: Sesión de 45min:

- 15min: Walkthrough del código
- 20min: Pair programming (mejoramos algo juntos)

- 10min: Discusión de arquitectura
-

Lo que realmente importa

Buscamos ingenieros que:

- Piensan en **problemas reales** (seguridad, escala, mantenibilidad)
- Documentan **por qué** tomaron decisiones
- Escriben código que **otros pueden mantener**
- Balancean **pragmatismo con calidad**

No buscamos código perfecto, buscamos **pensamiento de ingeniero**.

Éxito!